

When Process Credit Cannot Matter: Auditing Decision Authority in Verifier-Guided Repair

Adarsh Agrawal, Senior Member, IEEE

Abstract

Process credit is meant to help a coding agent keep promising intermediate states. That idea only works if the score can change a decision and if the state it prefers is more likely to verify later. We test both conditions in `BRANCHCREDIT`, a width-2 repair controller for Lean and Dafny. Its commitment rule places scalar credit after native verification, an integrity check, and five diagnostic fields. In 216 Lean decisions with multiple candidates, 53 contain a latent disagreement involving credit, but earlier fields settle every choice. Removing credit therefore changes 0/216 selected branches. We then move credit directly below the two hard gates. This promoted rule changes 54/216 selections and 36/144 episode paths, yet changes 0/144 final verification outcomes on the same recorded candidates. A smaller live replay regenerates descendants after forcing the credit-preferred branch: two of five disagreements improve, one worsens, and two do not change. Matched end-to-end evaluations also fail to establish an advantage. On the harder Lean set, `BRANCHCREDIT` solves 29/48 tasks and a branching-matched credit-free controller solves 27/48 (+4.2 percentage points, source-level $p = 0.50$). Other Lean and Dafny comparisons include small positive, exact-null, and negative differences. First-call predictors that look strong on development sources fall toward chance on held-out sources. The result is a practical design test: a process score needs measurable decision authority and held-out continuation value. Without both, it is either unused or unreliable.

Introduction

Native verifiers give coding agents feedback that ordinary tests cannot. A Lean proof must type-check, and a Dafny program must discharge its verification conditions. This has made theorem proving and verified code generation natural settings for search at inference time [7, 9, 11, 15, 17]. It also invites a simple idea: give partial credit to an intermediate repair that removes errors, reaches a new proof state, or avoids a repeated failure, then continue from the highest-scoring branch.

The score itself is not enough. It must pass two tests. *Decision authority* asks whether changing or deleting the score changes which branch the controller keeps. *Continuation value* asks whether the preferred branch is more likely to lead to a verified result. These are different failure modes. A score can be informative but placed too late in a lexicographic ordering to affect behavior. It can also affect behavior while tracking visible local progress rather than future success.

This paper studies both failures in `BRANCHCREDIT`, a test-time proof-repair controller around frozen hosted language models. At each round, `BRANCHCREDIT` generates two sibling

repairs, checks both, and commits one as the next incumbent. The scalar credit signal updates task-local strategy scores, but it is also the eighth field of a nine-field commitment key. Native verification and an integrity contract come first, followed by five diagnostic fields. The architecture therefore raises a basic question: does the score that motivates the method ever decide anything?

We answer that question before interpreting end-to-end accuracy. First, we remove credit while keeping the same candidates, branch width, budget, and all other ranking fields. Second, we promote credit above every soft diagnostic field while preserving verification and integrity as hard gates. Third, at a stratified sample of disagreements, we force the credit-preferred branch and generate new descendants. Finally, we compare terminal completion with a branching-matched credit-free controller and with sequential verifier-guided repair. Source theorems or program families, not repeated model calls, are the independent units.

The answer is clear at the mechanism level. At its original rank, credit changes no observed Lean commitment. After promotion, it changes one quarter of multi-candidate selections but no final outcomes on the recorded candidates. Live descendants show that some preferences are consequential, but their direction is mixed. The terminal comparisons agree with that diagnosis: no frame establishes a reliable advantage for the credit signal.

Our contributions are:

- a simple proposition and complete decision audit for identifying when a late process score is unable to affect a lexicographic controller;
- a soundness-preserving promotion test that separates branch changes, path changes, and final verification changes;
- matched Lean and Dafny comparisons that isolate scalar credit from branch width and report uncertainty at the source level; and
- a held-out analysis showing that the available first-call features do not support a selective credit policy.

We do not claim that process credit is generally useless, that branching is harmful, or that the compared controllers are equivalent. The narrower finding is that this credit signal has no authority at its designed rank and no demonstrated terminal value after it is given authority.

Related Work

Process supervision and inference-time search. Process supervision gives feedback to intermediate reasoning steps rather than only final answers [1]. Tree search and test-time scaling use such signals to decide where additional inference

should be spent [3, 5]. **BRANCHCREDIT** studies a smaller but common engineering pattern: complete-file repair using frozen models, native checking after each call, and task-local scores that are discarded after the task. Its strategy selection uses upper-confidence bookkeeping related to classical bandit methods [19]; our question is whether that bookkeeping reaches the final commitment decision.

Theorem proving and verified code. LeanDojo, DeepSeek-Prover, and LeanProgress use proof-assistant interaction, search, retrieval, or progress prediction to improve theorem proving [7, 9, 11]. AxDafny and Meta-Step Decoding study agentic generation and verifier-guided translation in Dafny [15, 17]. These systems make verifier feedback central to search. Our contribution is not a new prover. It is an audit of the controller that turns several verifier and process signals into one branch choice.

Verifier quality. A native checker certifies only the contract it is given. It may not capture the full intent of a task, and a reward can exploit omissions in the verifier or integrity policy [13]. For that reason, **BRANCHCREDIT** keeps verification and integrity above all soft scores. The problem studied here appears inside that soundness-first design: additional diagnostic fields can also make the nominal process score irrelevant.

BranchCredit

Repair loop

A task is $\mathcal{T} = (x_0, D, P, V, B)$, where x_0 is a failing source file, D is the required declaration set, P is a protected-source contract, V is the native verifier, and B is the call budget. The controller chooses among six repair strategies, including local repair, multi-error repair, proof reconstruction, structural reasoning, rewriting, and restart. It requests two sibling files from the same incumbent and checks both before committing the next state. With six model calls, width two gives three branch rounds.

The integrity check rejects changed theorem signatures, protected-source edits, new axioms or unsafe declarations, placeholders, missing declarations, and disallowed imports or commands. A candidate is counted as solved only when both integrity and native verification pass.

Scalar credit combines verified success, diagnostic progress, removed errors and placeholders, diagnostic novelty, repeated-failure penalties, and checker or API cost. It updates the controller’s task-local strategy scores. The important detail is where it appears in the commitment key.

Three commitment rules

For candidate x , define

$$H(x) = (v, i, p, -e, -h, n, -r),$$

where v is verified success, i is integrity, p is diagnostic progress, e is the error count, h is the placeholder count, n is diagnostic novelty, and r is the repeated-failure count. Let $c(x)$

be scalar credit and $j(x)$ the call index. We compare:

$$K_{BC}(x) = (H(x), c(x), -j(x)), \quad (1)$$

$$K_{CF}(x) = (H(x), -j(x)), \quad (2)$$

$$K_{\text{promoted}}(x) = (v, i, c, p, -e, -h, n, -r, -j). \quad (3)$$

CREDITFREE removes only scalar credit. It keeps the same parent, candidates, branch width, calls, checks, stopping rule, and non-credit fields. The promoted rule moves credit directly below verification and integrity. It is allowed to choose among soundness-equivalent unverified candidates, but it cannot override a verified candidate or one that passes integrity.

When a score cannot matter

Proposition 1 (Decision authority under lexicographic commitment). *Fix a candidate pool, deterministic final tie-breaking, and a ranking $K_c = (H, c, t)$, where every field of H precedes process credit c . Removing c can change the selected candidate only if at least two candidates tie for the maximum value of H and c changes the winner selected by (H, t) . If H has a unique maximizer at every reachable decision, removing c leaves the recorded trajectory unchanged. If credit acts only among candidates with the same verification status, it cannot change the immediate binary verification result.*

Proof. Lexicographic comparison consults c only after equality on every field of H . A unique maximizer of H is therefore selected with or without c . Applying the same argument at each reachable decision gives trajectory invariance for a fixed call schedule and candidate stream. Selecting among candidates with identical verification status cannot change that status at the current decision. $\square$

The proposition gives a direct audit. We count where credit has a different preference, where the ranking actually reaches credit, where the selected branch changes, and where final verification changes. Keeping these quantities separate prevents score variation from being mistaken for control.

Evaluation

Tasks and experimental frames

The controlled Lean tasks begin with verified source theorems. We replace each proof body with a unique invalid term while preserving its signature. Every reference file must verify and every mutation must fail before evaluation. The main mechanism set has 12 independent source families. Four hosted model families and three prompt seeds produce 144 **BRANCHCREDIT** episodes and 432 branch rounds. Of those rounds, 216 contain multiple checked candidates and can support a selection comparison.

We use four additional settings:

- a harder Lean set with 12 source families and 48 episodes per controller;
- a separate near-ceiling Lean set with 12 source families and 72 episodes per controller;
- an external Lean set with 12 source families and 96 episodes each for **BRANCHCREDIT** and **VERIFIERGUIDED**; and

- two 12-program Dafny sets, one arithmetic and one invariant-heavy, with 48 episodes per controller.

The controlled comparisons use six model calls, six native checks, the same output limit, and matched prompts and decoding settings. We do not give one controller an early-stop budget advantage. The near-ceiling CREDITFREE arm was executed separately under the same budget, so we treat it as a matched comparison rather than part of one simultaneous multi-controller run.

Models were accessed through AWS Bedrock during July–August 2026: Claude Sonnet 5, Claude Haiku 4.5, Anthropic Fable 5, GPT-5.6 Terra, GPT-5.6 Luna, Qwen3 Coder 480B, and DeepSeek V3.2. The Anthropic, Qwen, and DeepSeek models used Bedrock Runtime; the GPT models used Bedrock Mantle. Controlled frames used matched prompt construction and an 8,192-token output ceiling. Sampling controls were interface-specific, so we do not report a universal temperature or top- p value. Labels 17, 43, and 101 index experimental conditions. Appendix gives the model subset for each frame.

For a natural-data check, we mine 27 paired proof-maintenance units from batteries (24 units) and aesop (3 units), both Apache-2.0. A unit is admitted when the evaluated revision compiles cleanly and the mutation produces a valid evaluation input that fails native Lean verification. Both arms use Claude Sonnet 5 through Bedrock Runtime in us-west-2 during August 2026, with a matched six-call budget per arm. Only frontier selection differs between BRANCHCREDIT and CREDITFREE. Exact evaluated revisions are recorded in the supplementary artifact.

A secondary Mathlib-30 frame evaluates 30 proof-repair units with Claude Sonnet 5 and Claude Haiku 4.5 under three-call budgets. It compares the branching BRANCHCREDIT policy with sequential VERIFIERGUIDED. Because that comparison changes branching, parent selection, and feedback timing in addition to credit, we report it only as a policy-level stress test, not as an isolation of scalar credit.

Interventions and outcomes

The original-rank and promoted-rank analyses replay the same checked candidate pools. This removes generation noise and asks a deterministic question: would a different ranking choose a different recorded state? The live replay then tests the downstream question. A point is eligible when both siblings were generated and checked. We sample 20 of 80 eligible points across all 12 source families, force the credit-preferred sibling when it differs from the original choice, and regenerate the remaining descendants.

The main mechanism quantities are:

1. multi-candidate decisions;
2. latent credit disagreements;
3. selected-branch changes;
4. changed episode paths; and
5. changed final verification outcomes.

The main terminal outcome is verified completion. CREDITFREE is the primary baseline because it isolates the scalar credit field. VERIFIERGUIDED is a sequential baseline that generates one repair per call and returns to the best verifier-ranked state.

Inference and prediction

The source theorem or program family is the independent unit. We aggregate model and prompt repetitions within a source, use source-cluster bootstrap intervals, and report exact source-level sign-flip tests. Paired McNemar tests at the episode level are descriptive. We never read a failure to reject as evidence of equivalence [21, 23, 25].

The predictor analysis uses 144 episodes over the same 12 Lean source families, two models, three seeds, and two controllers with comparable intermediate states. Eight source families form the development split and four size-stratified families are held out. Features use first-call information only: error reduction, diagnostic progress, scalar credit, and proposal entropy. We report AUC on all held-out episodes and again after excluding episodes already verified on the first call.

Results

The causal funnel

Figure 1 shows the complete mechanism result. At the original rank, fields one through four settle 21 of 216 multi-candidate commitments. Another 142 decisions compare a pool containing a verified candidate. The remaining 53 are the meaningful latent-disagreement set: the candidates are unverified and scalar credit prefers a different candidate from the credit-free choice. Yet placeholder count, novelty, or repeated-failure count settles all 53 before the ranking reaches credit. Removing credit changes 0/216 selected branches and 0/144 final outcomes.

Promotion rules out a purely architectural excuse. With credit at field three, 54/216 selections change. All 54 are between integrity-passing, unverified candidates. The committed path changes in 36/144 episodes, but BRANCHCREDIT, CREDITFREE, and promoted credit each solve 81/144 tasks. No episode changes final verification. Credit now has authority, but the recorded preference does not improve the outcome.

Live replay weakens the fixed-candidate assumption. Among the 20 sampled eligible points, five have a different credit preference. Forcing that branch and regenerating descendants converts two failures to successes, one success to a failure, and leaves two unchanged. Conditional on a disagreement, 3/5 outcomes are consequential, with Wilson 95% interval [0.231, 0.882]. The net success difference is +1/5, and the exact sign test is $p = 1.0$. The sample is too small to claim benefit, but it shows that credit is not always harmless once it is allowed to control the path.

Terminal completion

Figure 2 reports the matched terminal comparisons. The harder Lean set is the most favorable result for BRANCHCREDIT: 29/48 tasks verify, compared with 27/48 for CREDITFREE. The difference is +4.2 percentage points, the source-cluster interval is [0.0, 10.4], and the exact source-level test is $p = 0.50$. This is a positive point estimate, not evidence of superiority.

The separate near-ceiling Lean set gives one additional solve for BRANCHCREDIT: 60/72 versus 59/72 for both CREDITFREE and VERIFIERGUIDED, with $p = 1.0$. The external Lean set runs in the other direction: BRANCHCREDIT solves 58/96 and

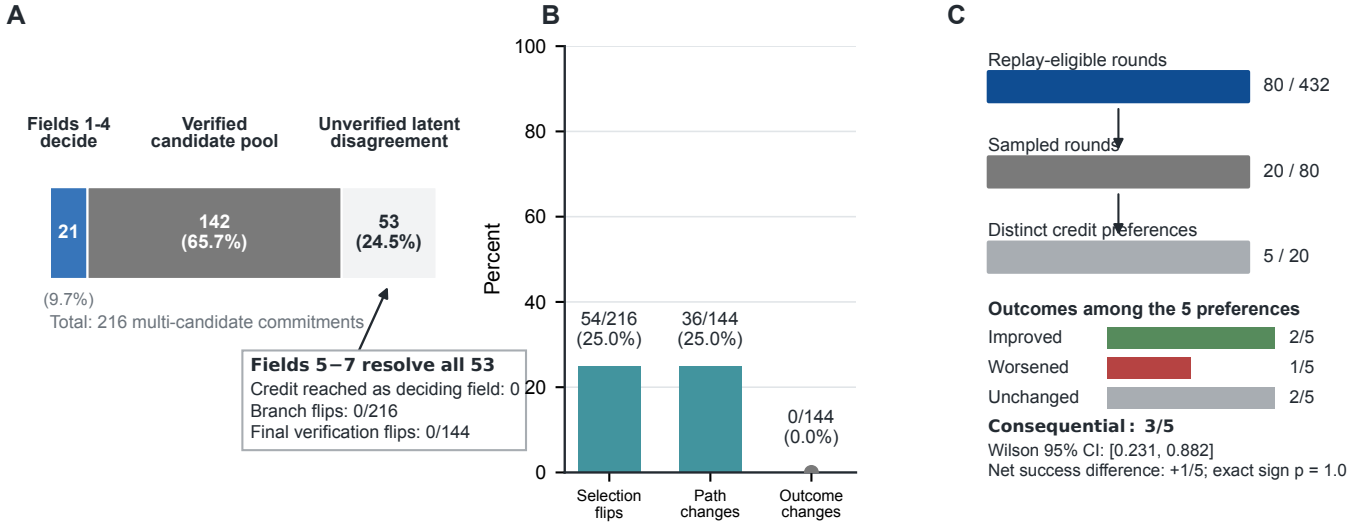


Figure 1: Decision authority and continuation value. **A:** At the original rank, earlier fields settle every multi-candidate decision. The 53 unverified latent disagreements never reach credit. **B:** Promoting credit changes 54/216 selections and 36/144 episode paths, but no final verification outcome on the same candidates. **C:** In live replay, five sampled states have a distinct credit preference. Forcing it improves two outcomes, worsens one, and leaves two unchanged. The interval and sign test are conditional on these five states and are not population estimates for all 432 rounds.

VERIFIERGUIDED solves 67/96, a -9.4 -point difference with interval $[-19.8, 0.0]$ and $p = 0.156$.

The Dafny replication reaches a similar conclusion through a different verifier. On arithmetic tasks, BRANCHCREDIT solves 46/48 and CREDITFREE solves 45/48. On the invariant-heavy set, both solve 48/48. The mechanism differs across the two Dafny sets. Credit ablation changes 0/144 arithmetic commitments, while in the invariant-heavy set scalar credit frequently decides among candidates that are already verified. In both cases, there is little room for scalar credit to change binary completion.

On the 27 natural proof-maintenance units, BRANCHCREDIT solves 14 and CREDITFREE solves 13. There is one discordant unit and exact McNemar $p = 1.0$. BRANCHCREDIT uses 94 model calls and 397k output tokens, compared with 103 calls and 440k tokens for CREDITFREE. It also reaches fewer distinct checked frontier states (3 versus 7) and records fewer recoveries after a wrong branch (2 versus 3). These process counts occur after the selected trajectories diverge, so we report them as description, not as a causal efficiency result.

Secondary Mathlib policy comparison. On Mathlib-30 with Claude Sonnet 5, BRANCHCREDIT solves 27/30 units and sequential VERIFIERGUIDED solves 30/30. With Claude Haiku 4.5, the corresponding results are 6/30 and 7/30. These comparisons do not isolate scalar credit from the other policy differences, so they provide external stress-test evidence rather than a causal credit estimate.

Held-out prediction

A selective controller would use credit only when early evidence predicts a good continuation. The development split makes that idea look promising: error reduction, diagnostic progress, and scalar credit each have AUC above 0.93. Figure 3 shows that the result does not transfer. Held-out AUC falls to 0.581–0.643

for those features and to 0.521 for proposal entropy. After first-call successes are removed, AUC is 0.49–0.61. No held-out source-level test rejects chance.

What the Result Changes

Audit authority before accuracy. A score distribution or correlation does not show that a controller uses the score. The cheapest decisive test is an ablation on the actual commitment rule. Here that test gives an exact zero: credit is never the deciding field at its designed rank.

Keep hard gates, shorten the soft prefix. Verification and integrity should remain above a heuristic score. Within those gates, however, a long diagnostic prefix can silently make the score decorative. A controller paper should publish the full ordering and report how often each field decides.

Local progress is not future value. Promoted credit changes one quarter of selections but no recorded final outcomes. Live replay gives both gains and harm. The score tracks visible movement, but it is not calibrated to the probability that the selected state admits a successful continuation.

Branching and credit need separate controls. BRANCHCREDIT and a sequential verifier-guided controller differ in branch width, parent selection, and budget allocation. Only BRANCHCREDIT VERSUS CREDITFREE isolates the credit signal. A point estimate against a sequential baseline cannot be attributed to credit alone.

A stronger successor is testable. The next version should learn or calibrate descendant success on development sources,

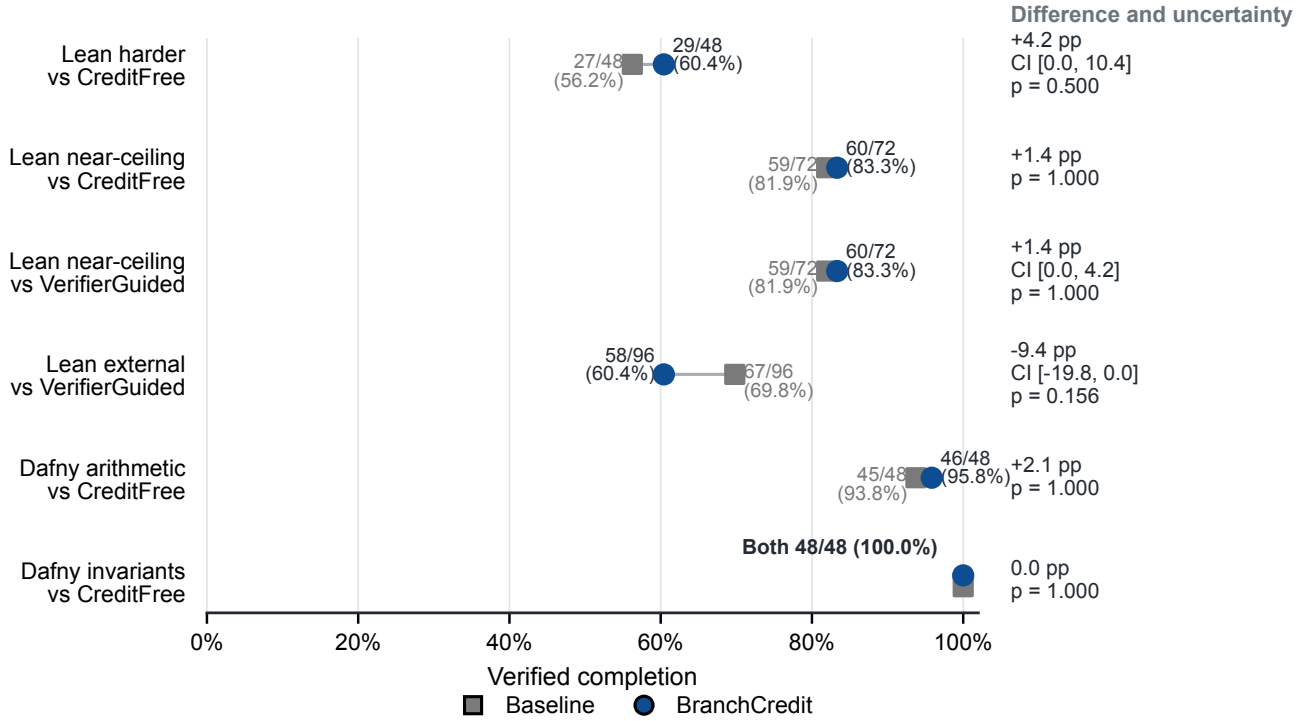


Figure 2: Verified completion under matched budgets. Gray squares are the named baseline and blue circles are BRANCHCREDIT. Intervals and tests aggregate repeated episodes within the source family. The credit-isolating comparisons are BRANCHCREDIT versus CREDITFREE; comparisons with VERIFIERGUIDED also change the branching policy. No contrast establishes a reliable advantage, and non-significance is not an equivalence result.

validate on untouched source families, and compare selective credit with always-credit and never-credit controls. It should also declare an authority target, such as the fraction of soundness-equivalent decisions the value estimate is allowed to override.

Limitations

The original-rank audit is exact for the recorded candidates. The promoted analysis also uses those candidates, so it does not regenerate every trajectory after every changed choice. The live intervention addresses this issue at only 20 points, including five actual disagreements. That is enough to demonstrate mixed consequences, but not enough to estimate a stable improvement or harm rate.

The controlled Lean tasks come from 12 source families and use synthetic proof-body mutations. The natural evaluation contains only 27 units from two repositories and is not a representative sample of all proof maintenance. Neither setting covers large multi-file changes, specification repair, or long interactive developments. Dafny adds a second verifier, but both Dafny sets are near ceiling.

Hosted model calls may change as providers update models and serving systems. We match call and verification budgets and aggregate at source level, but exact regeneration may still drift. With 12 independent sources, the source-level tests have limited power. We therefore say that the experiments do not establish an advantage; we do not claim equivalence.

Finally, scalar credit is one engineered composite. The paper

does not show that all process-value functions fail. A value model trained directly on descendant success, or a controller with a different authority allocation, may behave differently.

Reproducibility and Impact

The anonymized supplementary artifact accompanying the submission preserves controller configurations, prompts, task schemas, recorded model responses, native verifier outputs, integrity decisions, branch traces, analysis scripts, figure data, and failed candidates. For third-party repositories, it records exact revisions and mutation metadata rather than redistributing reference proof bodies. Preparation scripts reconstruct the evaluated tasks from those revisions. Hosted regeneration may differ across provider runs, but the recorded responses and candidates support replay of the verifier and analysis stages without re-calling hosted models.

Controlled comparisons use Lean 4.33.0 (commit d8b18978) or Dafny 4.9.0 with Z3 4.15.4. Evaluation uses AWS EC2 compute instances running Ubuntu 22.04 LTS. The mechanism audit contains 144 episodes, 432 branch rounds, and 216 multi-candidate decision states under matched six-call budgets and 8,192-token ceilings. The harder Lean, near-ceiling, external, Dafny, and natural frames also use six-call budgets; Mathlib-30 uses three-call budgets. The supplementary artifact provides per-frame model rosters, hardware specifications, actual call counts, wall-clock times, token totals, and accounting for preliminary exploratory runs.

Verifier-grounded control can improve auditability because

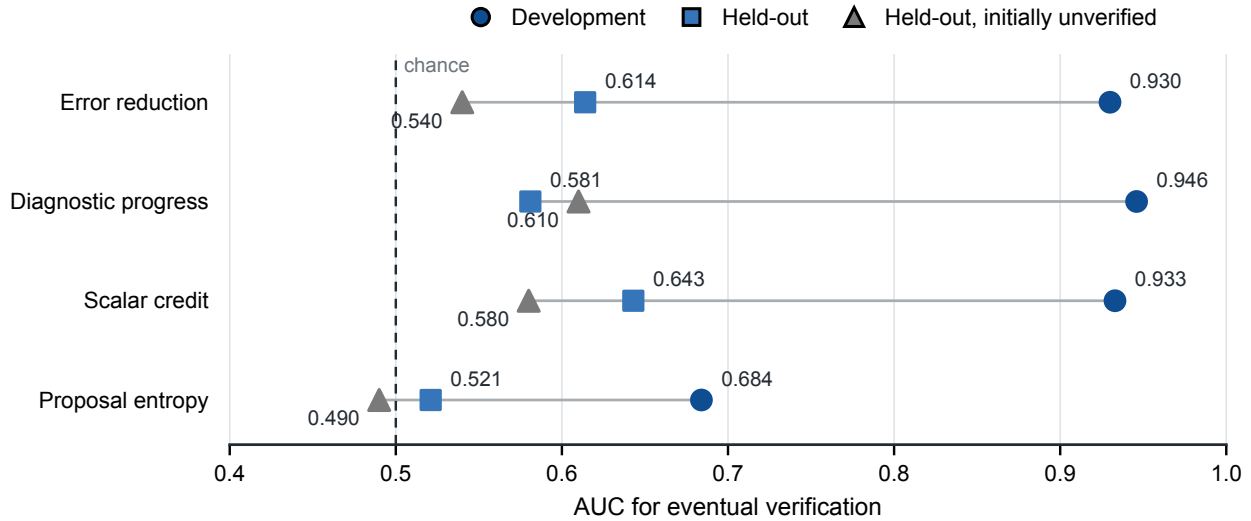


Figure 3: First-call prediction does not transfer cleanly. The development set contains eight source families and 96 episodes; the held-out set contains four source families and 48 episodes. Triangles exclude episodes already verified on the first call. Strong development AUC falls toward chance on held-out sources, so these features do not justify a selective credit policy.

accepted outputs and rejected candidates can be checked and preserved. It can also create false confidence when the verifier or integrity contract misses important behavior. Process scores add another risk: a precise number may be mistaken for a calibrated probability of success. Systems using these methods should expose their commitment rule, keep human review for consequential use, and avoid presenting diagnostic movement as correctness.

Lean 4.33.0, batteries, aesop, and mathlib4 are used under Apache-2.0; Dafny 4.9.0 and Z3 4.15.4 are used under MIT licenses. The supplementary artifact records exact repository revisions, the specific ITPEval source and its redistribution terms, and the provider routes for hosted models. No model weights are redistributed. Newly authored controller code is released under MIT, while task schemas, prompts, and result tables are released under CC-BY-4.0 subject to the terms of their upstream sources.

Conclusion

BRANCHCREDIT computes process credit, but its original commitment rule never lets that credit decide among the observed Lean candidates. Moving the score directly below verification and integrity creates substantial branch and path changes, yet no final change on the recorded candidates. Live descendants confirm that credit-preferred branches can matter, while also showing that the preference is not reliably directional.

The terminal evidence is consistent with the mechanism. It contains small positive, null, and negative differences, but no source-level result that establishes a credit advantage. A process score should therefore be evaluated as part of a controller, not in isolation. It needs measurable authority, a held-out relationship with continuation success, and a branching-matched signal-free baseline. Without those checks, extra scoring machinery can look meaningful while doing little useful work.

References

- [1] H. Lightman et al. Let’s Verify Step by Step. arXiv:2305.20050, 2023.
- [2] S. Indukuri and A. Agrawal. Grounded Optimization: A Layered Engineering Framework for Reducing LLM Hallucination in Automated Personal Document Rewriting. arXiv:2607.01457, 2026.
- [3] S. Yao et al. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. NeurIPS, 2023.
- [4] Amazon Artificial General Intelligence. The Amazon Nova Family of Models: Technical Report and Model Card. Amazon Technical Report, 2024. Amazon Science publication page.
- [5] C. Snell et al. Scaling LLM Test-Time Compute Optimally Can Be More Effective than Scaling Model Parameters. arXiv:2408.03314, 2024.
- [6] A. Agrawal and R. S. Babu. Incident Memory: Training-Free Operational Memory through Sequential Pattern Mining and Velocity-Stratified Retrieval. arXiv:2609.01616, 2026.
- [7] K. Yang et al. LeanDojo: Theorem Proving with Retrieval-Augmented Language Models. NeurIPS, 2023.
- [8] B. Vidgen, A. Agrawal, A. M. Ahmed, V. Akinwande, N. Al-Nuaimi, N. Alfaraj, et al. Introducing v0.5 of the AI Safety Benchmark from MLCommons. arXiv:2404.12241, 2024.
- [9] H. Xin et al. DeepSeek-Prover-V1.5: Harnessing Proof Assistant Feedback for Reinforcement Learning and Monte-Carlo Tree Search. arXiv:2408.08152, 2024.

- [10] Amazon Artificial General Intelligence. Amazon Nova Sonic: Technical Report and Model Card. Amazon Technical Report, 2025. Amazon Science publication page.
- [11] R. J. George, S. Huang, P. Song, and A. Anandkumar. LeanProgress: Guiding Search for Neural Theorem Proving via Proof Progress Prediction. arXiv:2502.17925, 2025.
- [12] A. Agrawal and S. Indukuri. Schema-First Retrieval: Embedding Catalogs for Natural Language Analytics. arXiv:2606.28387, 2026.
- [13] Qwen Team. The Verification Horizon: No Silver Bullet for Coding Agent Rewards. arXiv:2606.26300, 2026.
- [14] W. Feng, A. Agrawal, H. Ling, E. Blasch, E. Adiles-Cruz, P. T. Schrader, and J. Wei. DDDAS Probability Learning for Natural Disaster Change Detection. International Conference on Dynamic Data Driven Applications Systems, pp. 90–99, 2024. https://doi.org/10.1007/978-3-031-94895-4_9.
- [15] B. Breen, A. Letson, B. Requena Pozo, and L. Sarra. AxDafny: Agentic Verified Code Generation in Dafny. arXiv:2606.32007, 2026.
- [16] Amazon Artificial General Intelligence. Amazon Nova Premier: Technical Report and Model Card. Amazon Technical Report, 2025. Amazon Science publication page.
- [17] T. Zhou, S. Jha, M. Christodorescu, K. Levchenko, and V. Chandrasekaran. Verifier-Guided Code Translation via Meta-Step Decoding. arXiv:2605.17626, 2026.
- [18] R. S. Babu and A. Agrawal. Self-Healing Agentic Orchestrators for Reliable Tool-Augmented Large Language Model Systems. arXiv:2606.01416, 2026.
- [19] P. Auer, N. Cesa-Bianchi, and P. Fischer. Finite-Time Analysis of the Multiarmed Bandit Problem. Machine Learning, 47(2–3), 2002.
- [20] B. Vidgen, A. Agrawal, A. M. Ahmed, V. Akinwande, N. Al-Nuaimi, N. Alfaraj, et al. others. 2024. Introducing v0.5 of the AI Safety Benchmark from MLCommons. arXiv preprint arXiv:2404.12241, 2024.
- [21] Q. McNemar. Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages. Psychometrika, 1947.
- [22] Amazon Artificial General Intelligence. Amazon Nova Multimodal Embeddings: Technical Report and Model Card. Amazon Technical Report, 2025. Amazon Science publication page.
- [23] B. Efron. Bootstrap Methods: Another Look at the Jackknife. Annals of Statistics, 1979.
- [24] A. Agrawal and J. Li. Mitigating Bias in AI Using DebiGAN. NVIDIA GTC Conference, 2022.
- [25] I. A. Canay, J. P. Romano, and A. M. Shaikh. Randomization Tests Under an Approximate Symmetry Assumption. Econometrica, 2017.
- [26] Amazon Artificial General Intelligence. Amazon Nova 2: Multimodal Reasoning and Generation Models. Amazon Technical Report, 2025. Amazon Science publication page.
- [27] A. Agrawal and N. Garg. Time Series Forecasting of Producer Price Index, using ARIMA. International Journal of Science and Research, 7(7):1019–1024, 2018.
- [28] B. Vidgen, A. Agrawal, A. M. Ahmed, V. Akinwande, N. Al-Nuaimi, N. Alfaraj, et al. Introducing v0.5 of the AI Safety Benchmark from MLCommons, 2024. <https://arxiv.org/abs/2404.12241>.
- [29] B. Vidgen, A. Agrawal, A. M. Ahmed, V. Akinwande, et al. How to Construct AI Safety Benchmarks. AI safety benchmark technical presentation and associated benchmark-design work, 2023.

Exact Audit Definitions

The full current ranking is

$$K_{BC} = (v, i, p, -e, -h, n, -r, c, -j).$$

The credit-free ranking deletes only the eighth field. The promoted ranking is

$$K_{\text{promoted}} = (v, i, c, p, -e, -h, n, -r, -j).$$

All 54 promoted selection changes occur after both candidates pass integrity and before either candidate verifies.

Replay eligibility requires a multi-candidate round in which both siblings were generated and checked. There are 80 such rounds among the 432 mechanism rounds. The 20-point live sample is stratified over all 12 source families, with one or two points per family. Five sampled points have a distinct scalar-credit preference. The 3/5 consequentiality interval is conditional on those five points, not an estimate for the complete round population.

Additional Descriptive Results

Per-frame model and policy details

Table 1: Natural proof-maintenance comparison. Call and token counts are post-selection outcomes and are not interpreted as randomized efficiency effects.

Metric	BC	CF
Solved source units	14/27	13/27
Model calls	94	103
Output tokens	397k	440k
Distinct checked frontier states	3	7
Wrong-branch recoveries	2	3

Table 2: Hosted model subsets and call budgets by evaluation frame.

Frame	Hosted models	Calls
Mechanism and promoted replay	Claude Sonnet 5, GPT-5.6 Terra, Qwen3	6
Harder Lean	Coder 480B, DeepSeek V3.2	
	Claude Sonnet 5, GPT-5.6 Terra, Qwen3	6
	Coder 480B, DeepSeek V3.2	
Near-ceiling Lean	Claude Sonnet 5, GPT-5.6 Terra	6
External Lean	Claude Sonnet 5, Anthropic Fable 5, GPT-5.6 Terra, GPT-5.6 Luna	6
Held-out predictor	Claude Sonnet 5, GPT-5.6 Terra	6
Natural organic	Claude Sonnet 5	6
Dafny arithmetic and invariants	Claude Sonnet 5, GPT-5.6 Terra	6
Mathlib-30 secondary	Claude Sonnet 5, Claude Haiku 4.5	3

Table 3: Secondary Mathlib-30 policy comparison. This frame does not isolate credit from branching or sequential feedback.

Model	BC	Seq. VG
Claude Sonnet 5	27/30	30/30
Claude Haiku 4.5	6/30	7/30